PROOF OF CONCEPT

NYC Taxi Lakehouse

End-to-end ETL / ELT pipeline on Databricks

Medallion architecture · Unity Catalog · Delta Lake · Auto Loader · SCD Type 2 · Lakeflow Jobs · AI/BI Dashboard

Document type	Proof of Concept / Technical design document
Project	NYC Yellow Taxi analytics lakehouse
Platform	Databricks Free Edition (serverless)
Version	1.0
Date	8 September 2026
Status	Implemented and validated

Contents

1. Executive summary

This document describes a working proof of concept for an analytics lakehouse built on the New York City yellow taxi trip dataset. The system ingests files as they land in cloud storage, cleans and conforms them, models them as a dimensional star schema with slowly changing dimensions, and serves a dashboard that refreshes automatically whenever new data arrives.

The whole solution runs on Databricks Free Edition, which means serverless compute, Unity Catalog for governance, Delta Lake for storage, and Lakeflow Jobs for orchestration — no cluster administration and no cloud account of your own required. Every design choice in this document was made under that constraint and is stated explicitly, including the three places where the Free Edition forced a different approach from what a production deployment would use.

1.1 What was built

- A four-layer medallion pipeline: landing → bronze → silver → gold, each layer with a distinct and defensible responsibility.
- Incremental ingestion using Auto Loader, so files are never read twice and the pipeline can be rerun safely at any point.
- A data quality framework with fifteen rules that quarantines rejected records with their failure reasons instead of silently discarding them.
- A star schema with one fact table and five dimensions, including a full SCD Type 2 implementation on the taxi zone dimension.
- A Lakeflow job with a file-arrival trigger, a dependency graph across six tasks, and a data quality gate that fails the run rather than publishing bad numbers.
- An AI/BI dashboard with two pages, reading pre-aggregated gold marts that are rebuilt on every run.

1.2 Headline results

Dimension	Result	Notes
Volume processed	~1.8 M trip records	14 days of history at 120,000 trips/day, plus incremental days
Data quality pass rate	~91 %	Remainder quarantined with reasons, not dropped
Pipeline stages	6 orchestrated tasks	Plus a separate weekly maintenance job
End-to-end latency	Minutes from file arrival	File-arrival trigger, not a fixed schedule
Idempotency	Fully re-runnable	Every write is a MERGE on a stable key
History preservation	SCD Type 2 verified	Zone renames produce new versions; old trips keep old labels

2. Business context and objectives

The NYC Taxi and Limousine Commission publishes trip records for every licensed yellow taxi journey in New York City. Each record carries pickup and drop-off timestamps, location zone identifiers, passenger and distance measures, and a full fare breakdown. It is a genuinely awkward dataset: high volume, arriving as periodic file drops, dirty in predictable ways, and dependent on a reference lookup (the taxi zone table) that changes underneath you.

That combination is why it makes a good proof of concept. It exercises every part of a lakehouse rather than just the happy path.

2.1 Objectives

#	Objective	How it is met
O1	Ingest files from cloud storage without reprocessing what has already been loaded	Auto Loader with a checkpointed file registry
O2	Guarantee correctness under retries, replays and partial failures	Delta Lake ACID transactions; MERGE on stable keys
O3	Separate raw fidelity from business-ready data	Bronze keeps everything; silver applies rules
O4	Make rejected data explainable	Quarantine table with a per-row failure array
O5	Model the data for analysts, not engineers	Kimball star schema with conformed dimensions
O6	Preserve history when reference data changes	SCD Type 2 on dim_zone with point-in-time joins
O7	Refresh reporting automatically on new data	File-arrival trigger driving a Lakeflow job
O8	Govern access centrally	Unity Catalog catalog / schema / volume hierarchy with grants

2.2 Scope

In scope

- Yellow taxi trip records and the taxi zone reference lookup.
- Batch and micro-batch incremental ingestion, transformation, modelling and serving.
- Orchestration, data quality gating, table maintenance and dashboarding.

Out of scope

- Green taxi, for-hire vehicle and high-volume for-hire vehicle datasets — the same pipeline extends to them without structural change.
- Real-time sub-second streaming; the workload is file-driven and micro-batch is the right fit.
- Machine learning models. The gold layer is a suitable feature source but no model is built here.
- Production CI/CD. An asset bundle definition is provided but no build pipeline is configured.

3. Technology stack

Every component below is native to the Databricks platform. Nothing external needs to be installed, licensed or maintained, which is a large part of why the lakehouse pattern is attractive for a project of this shape.

Technology	Role in this project	Why it was chosen
Databricks Free Edition	Compute and workspace	Serverless — no cluster sizing, no cloud account, and it includes Unity Catalog, Jobs and AI/BI dashboards
Unity Catalog	Governance layer: catalog, schemas, volumes, grants, lineage	One permission model across tables, files and dashboards; three-level namespace makes the medallion layers explicit
Delta Lake	Storage format for every table	ACID transactions, schema enforcement and evolution, time travel, MERGE, change data feed
Auto Loader (cloudFiles)	Incremental file ingestion into bronze	Tracks processed files in its own checkpoint, so exactly-once ingestion comes for free and scales to millions of files

Technology	Role in this project	Why it was chosen
Structured Streaming	Bronze → silver micro-batches	Trigger.AvailableNow gives streaming semantics with batch economics — it starts, drains and stops
Spark SQL / PySpark	All transformation logic	SQL for set-based work such as the SCD2 merge, PySpark where control flow is needed
Liquid clustering	Physical layout of large tables	Multi-column data skipping that self-tunes file sizes and can be changed later without rewriting the table
Lakeflow Jobs	Orchestration	Task dependency graph, file-arrival triggers, retries and failure notification, all inside the workspace
AI/BI Dashboards	Serving layer	Reads gold marts directly with no BI tool to license or connect
Databricks Asset Bundles	Optional deployment as code	Job definitions live in version control alongside the notebooks

3.1 How the core pieces actually work

Delta Lake

A Delta table is a directory of Parquet files plus a transaction log. Every write appends a new JSON commit to that log describing which files were added and removed. Readers resolve the log to a consistent snapshot, which is what makes concurrent reads and writes safe: the dashboard can query the fact table while the job is merging into it and will simply see the version that was current when its query started.

Three consequences matter for this pipeline. Writes are atomic, so a failed job never leaves a half-loaded table. Old versions remain addressable, so DESCRIBE HISTORY gives a complete audit trail and RESTORE reverses a bad load. And MERGE is a first-class operation, which is what makes every stage of this pipeline idempotent.

Auto Loader

Auto Loader is a Structured Streaming source that watches a directory. It maintains its own record of files it has already ingested inside the stream checkpoint, using RocksDB for scale. When the job runs again, it lists the directory, subtracts everything it has already seen, and processes only the remainder. Dropping the same file back in twice does not produce duplicate rows in bronze; a job that crashes halfway resumes from where it stopped.

Schema inference and evolution are handled through a schema location. New columns appearing in an upstream file are added to the target rather than causing a failure, which matters because the TLC has changed the yellow taxi schema several times.

Unity Catalog

Unity Catalog gives a three-level namespace — catalog, schema, table — and extends the same governance to file paths through external locations and volumes. In this project the catalog is nyc_taxi and the schemas are landing, bronze, silver and gold, so the layer a table belongs to is visible in its fully qualified name. Column-level lineage across the whole pipeline is captured automatically.

4. Solution architecture

Data moves left to right through four zones. Each zone has exactly one job, and no zone reaches past its neighbour.

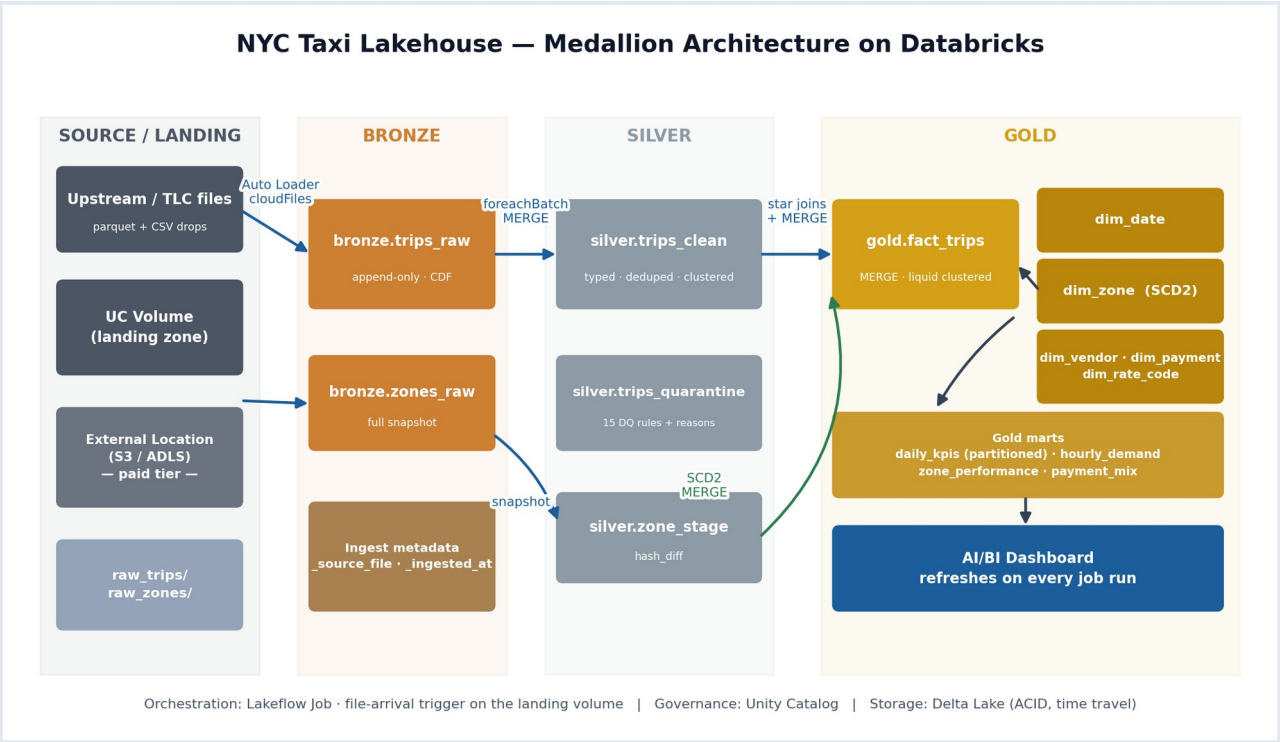


Figure 1 — End-to-end medallion architecture

4.1 Layer responsibilities

Layer	Contains	Rules it obeys
Landing	Raw parquet trip batches and CSV zone snapshots in a Unity Catalog volume	Write-only from the pipeline's point of view. Files are never modified or deleted by the job
Bronze	trips_raw, zones_raw	Append only. No filtering, no casting, no deduplication. Ingestion metadata added. This is what makes replay possible
Silver	trips_clean, trips_quarantine, zone_stage	Typed, conformed, deduplicated. Every rejected row is written to quarantine with its reasons
Gold	fact_trips, five dimensions, five marts	Business-ready star schema and pre-aggregated marts. The only layer analysts and dashboards touch

4.2 Why the raw layer is left untouched

The most common failure in a data platform is discovering, months later, that a cleaning rule was wrong. If bronze has been filtered, the evidence is gone and the only recovery is to ask the source system for history it may no longer have. Because bronze here is append-only and complete, fixing a rule means truncating silver, clearing one checkpoint and rerunning — a ten-minute operation described in the runbook.

5. Data model

5.1 Source schema

The generator produces the official TLC yellow taxi layout, so the pipeline can be pointed at real TLC parquet files without code changes.

Source column	Type	Meaning
`VendorID`	int	Technology provider that supplied the record
`tpep_pickup_datetime`	timestamp	Meter engaged
`tpep_dropoff_datetime`	timestamp	Meter disengaged
`passenger_count`	int	Driver-entered, therefore unreliable
`trip_distance`	double	Miles reported by the taximeter
`RatecodeID`	int	Final rate code in effect
`store_and_fwd_flag`	string	Y when the record was held in vehicle memory before transmission
`PULocationID` / `DOLocationID`	int	TLC taxi zone identifiers — the join to dim_zone
`payment_type`	int	Payment method code
`fare_amount` … `total_amount`	double	Full fare breakdown including surcharges and tips

5.2 Target star schema

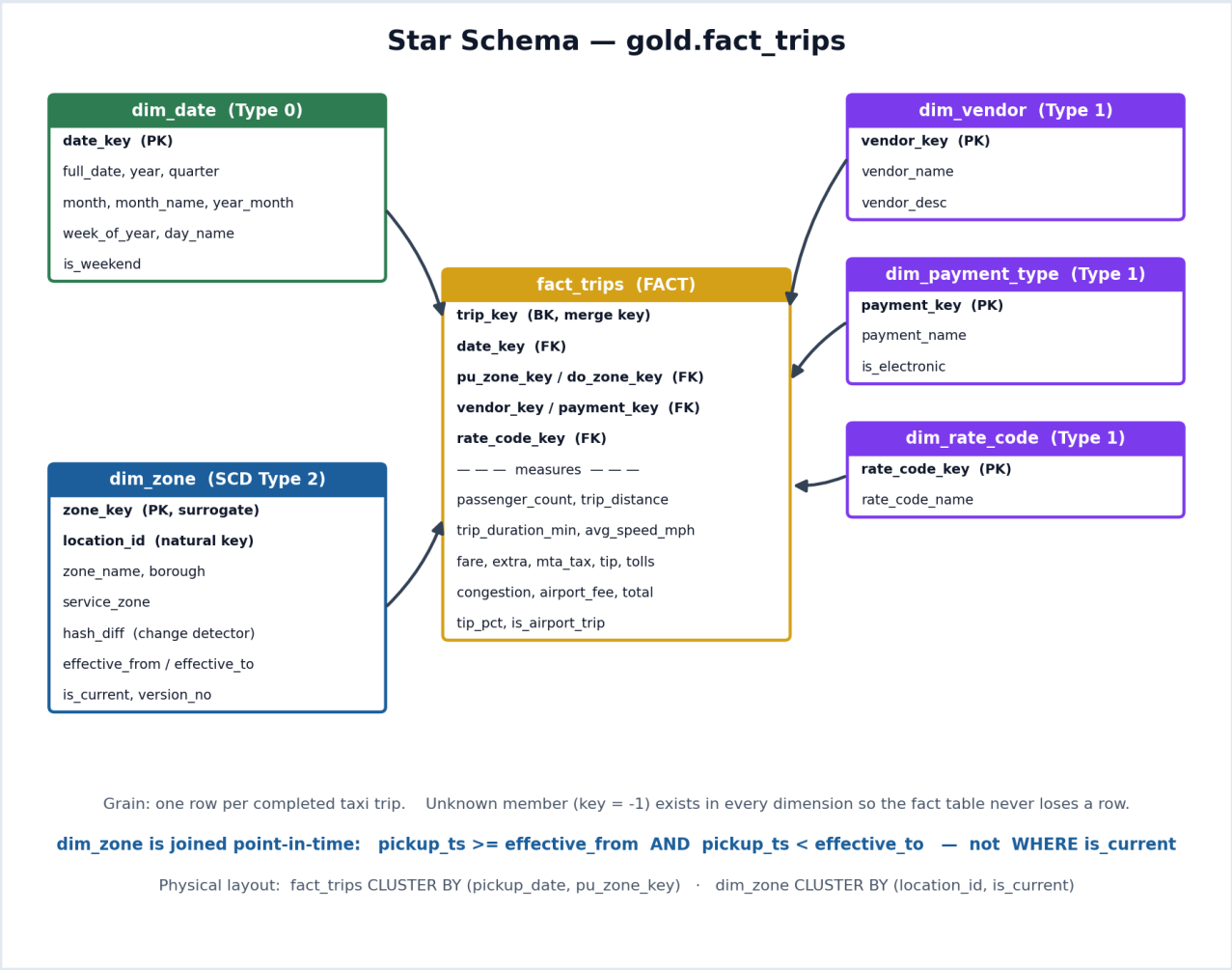


Figure 2 — Star schema: one fact, five dimensions

The grain of fact_trips is one row per completed taxi trip. Fixing the grain first is what keeps the model coherent; every measure in the table is additive at that grain, and anything that is not additive (tip percentage) is stored as a ratio and averaged rather than summed.

Dimension	SCD type	Reasoning
dim_date	Type 0	A calendar cannot change. Generated once for 2024–2027
dim_zone	Type 2	Zones get renamed and reassigned between service zones. A report for last March must use last March's labels
dim_vendor	Type 1	A short, fixed code list. Nobody asks what vendor 2 was called last year
dim_payment_type	Type 1	As above
dim_rate_code	Type 1	As above

Every dimension carries an Unknown member with surrogate key −1. Late-arriving or invalid foreign keys resolve to it, so the fact table never loses a row to a failed lookup and the orphan count is a metric rather than a silent data loss.

6. Detailed working of the pipeline

6.1 Landing zone and external locations

In a production deployment the landing zone is a path in your own object storage, governed by Unity Catalog through a storage credential and an external location:

```
CREATE STORAGE CREDENTIAL nyc_taxi_cred
  WITH IAM ROLE 'arn:aws:iam::<account>:role/databricks-uc-nyc-taxi';

CREATE EXTERNAL LOCATION nyc_taxi_landing
  URL 's3://my-datalake/nyc-taxi/landing/'
  WITH (STORAGE CREDENTIAL nyc_taxi_cred);

GRANT READ FILES, WRITE FILES ON EXTERNAL LOCATION nyc_taxi_landing TO `data_engineers`;
GRANT CREATE EXTERNAL TABLE ON EXTERNAL LOCATION nyc_taxi_landing TO `data_engineers`;
```

Free Edition has no cloud account behind it, so there is nothing for a storage credential to authenticate to and the statement above cannot succeed. The project therefore uses a Unity Catalog volume as the landing zone. Functionally this is the same thing for the pipeline: a governed path that files are written into and Auto Loader watches. Notebook 02 carries the full external-location and external-table syntax for both AWS and Azure; moving to a paid workspace is a one-line change to the landing path in the configuration notebook.

Managed versus external tables

Aspect	Managed (used here)	External
Storage location	Unity Catalog default storage	Your own bucket or container
DROP TABLE	Removes data after the retention window	Removes metadata only; files remain
Optimization	Predictive optimization and auto-compaction	You schedule OPTIMIZE yourself
Choose when	New pipelines; you want the platform to own lifecycle	Other engines read the same files, or residency is dictated by the bucket

6.2 Bronze — incremental ingestion

Auto Loader reads new files and writes them straight through to a Delta table, adding only ingestion metadata. Trigger.AvailableNow means the stream starts, processes everything currently available and stops, which suits a triggered job and avoids paying for idle compute.

```
bronze_stream = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "parquet")
    .option("cloudFiles.schemaLocation", f"{SCHEMA_STORE}/trips")
    .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
    .load(RAW_TRIPS)
    .select("*",
        F.col("_metadata.file_path").alias("_source_file"),
        F.current_timestamp().alias("_ingested_at")))

(bronze_stream.writeStream
    .option("checkpointLocation", f"{CHECKPOINT}/bronze_trips")
    .option("mergeSchema", "true")
    .trigger(availableNow=True)
    .toTable(T_BRONZE_TRIPS))
```

The zone lookup is handled differently. It arrives as a complete snapshot rather than a delta, so it is read as a batch and overwritten in bronze. Working out what actually changed is the SCD2 layer's job, not the ingestion layer's.

6.3 Silver — cleaning, conforming, deduplicating

Silver reads bronze as a stream and processes each micro-batch through four steps: derive a business key, score every row against the rule set, route failures to quarantine, and merge the survivors.

The business key

```
trip_key = sha2(concat_ws('||',
    VendorID, tpep_pickup_datetime, tpep_dropoff_datetime,
    PULocationID, DOLocationID, trip_distance, total_amount), 256)
```

File-based ingestion has one dominant failure mode: the same file being delivered twice. A deterministic hash over the natural attributes of a trip gives a key that survives re-delivery, so the MERGE that follows collapses duplicates whether they arrive in the same batch or a week apart.

The rule set

Fifteen rules are evaluated per row and collected into a dq_failures array. An empty array means the row is clean; anything else routes it to quarantine along with the reasons and the source file it came from.

Category	Rules	Typical cause
Completeness	null_timestamp, null_location	Meter transmission errors
Temporal logic	dropoff_before_pickup, duration_too_short, duration_too_long	Clock drift, meter left running
Range	distance_too_short, distance_too_long, total_out_of_range, implausible_speed	GPS drift, data entry errors
Business rules	invalid_passenger_count, too_many_passengers, negative_fare, non_positive_total	Voided trips, driver entry mistakes
Referential	pickup_date_too_old, pickup_date_in_future	Backfill contamination

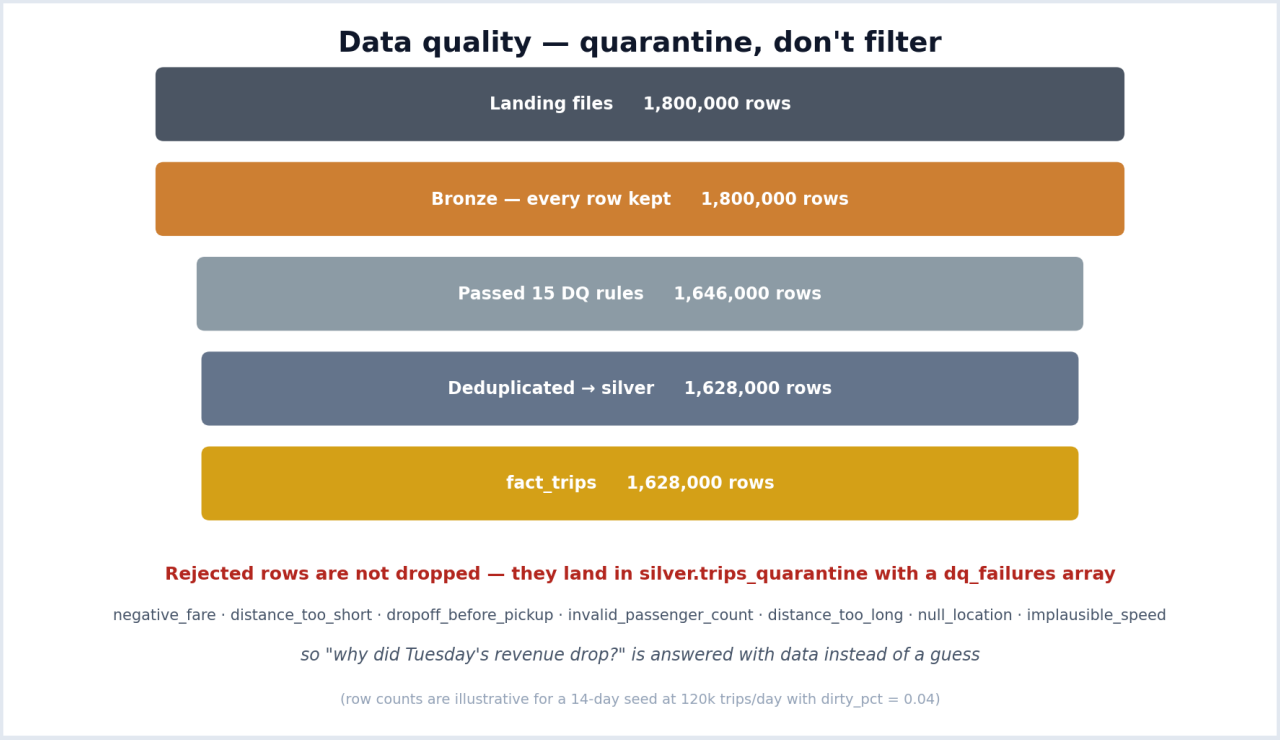


Figure 3 — Data quality funnel: rows are quarantined, never silently dropped

The distinction matters more than it first appears. A WHERE clause that removes negative fares makes the number look right and destroys the evidence. A quarantine table lets you answer why Tuesday's revenue dropped with a query rather than a guess, and it makes the pass rate a monitorable metric — the quality gate in task six fails the run if it drops below 85 percent.

6.4 Dimensions and the SCD Type 2 implementation

Change detection uses a hash of the tracked attributes rather than column-by-column comparison:

```
hash_diff = sha2(concat_ws('||', borough, zone_name, service_zone), 256)
```

One comparison replaces N, and adding a tracked attribute is a one-line change. The load itself is a single MERGE, but the source is deliberately fed in twice.

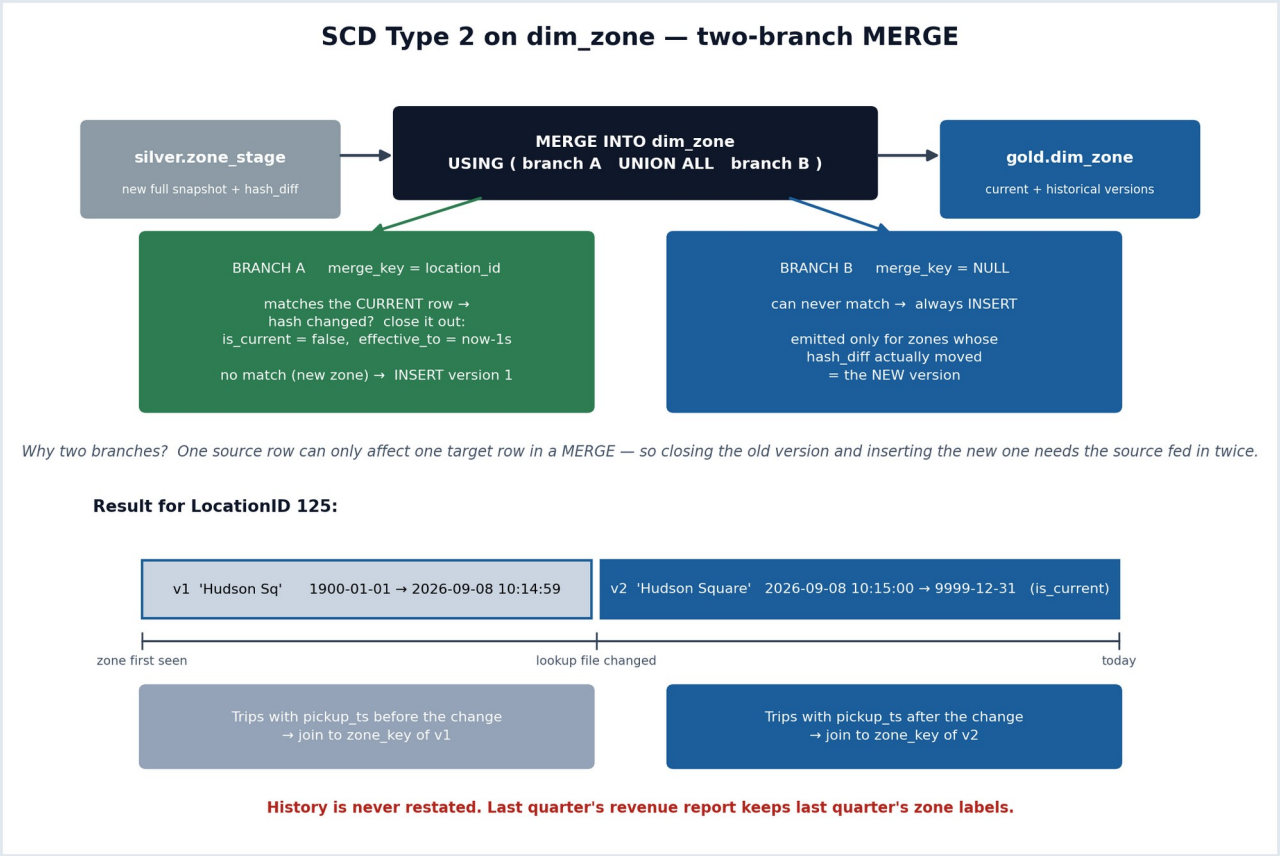


Figure 4 — The two-branch MERGE and its effect on history

The reason for two branches is a constraint of the MERGE statement itself: one source row can affect at most one target row. Closing the outgoing version and inserting the incoming one are two separate effects on two separate rows, so they need two separate source rows.

- Branch A sets merge_key to the natural key. It matches the current row. If the hash has changed, the row is closed out: is_current becomes false and effective_to is stamped one second before now. If nothing matches — a genuinely new zone — it falls through to the INSERT clause and becomes version 1.
- Branch B sets merge_key to NULL, which can never match anything, so it always inserts. It is emitted only for zones whose hash actually moved, and it carries the new version.

```
MERGE INTO gold.dim_zone AS t
USING ( branch_A  UNION ALL  branch_B ) AS src
   ON t.location_id = src.merge_key AND t.is_current = true

WHEN MATCHED AND t.hash_diff <> src.hash_diff THEN UPDATE SET
     t.is_current   = false,
     t.effective_to = current_timestamp() - INTERVAL 1 SECOND

WHEN NOT MATCHED THEN INSERT (...)
  VALUES (src.new_zone_key, src.location_id, ..., true, src.next_version, ...)
```

One detail worth calling out

Version 1 rows are opened at 1900-01-01 rather than at load time. Without this, trips already loaded for historical dates would fall outside every validity window the first time a zone changed, and the point-in-time join in the fact table would send them all to the Unknown member. It is the kind of bug that only shows up on the second run.

6.5 Fact table

The fact load reads only rows newer than a watermark, swaps every natural key for a surrogate key, and merges the result. The zone lookups are point-in-time rather than current:

```
LEFT JOIN gold.dim_zone pz
       ON pz.location_id = s.pu_location_id
      AND s.pickup_ts >= pz.effective_from
      AND s.pickup_ts <  pz.effective_to
```

Filtering on is_current instead would restate history every time a zone was renamed, quietly changing numbers in reports that had already been published. This join is the entire point of having built a Type 2 dimension.

6.6 Gold marts

Five small aggregate tables are rebuilt in full on every run: daily KPIs, hourly demand, zone performance, payment mix and a vendor scorecard, plus a pipeline health view that drives the freshness tiles. They are small enough that a full rebuild is cheaper and simpler than incremental maintenance, and it guarantees the dashboard is always internally consistent.

7. Incremental design and ACID guarantees

Three different incremental mechanisms are used, one per layer, because each layer has a different notion of what "new" means.

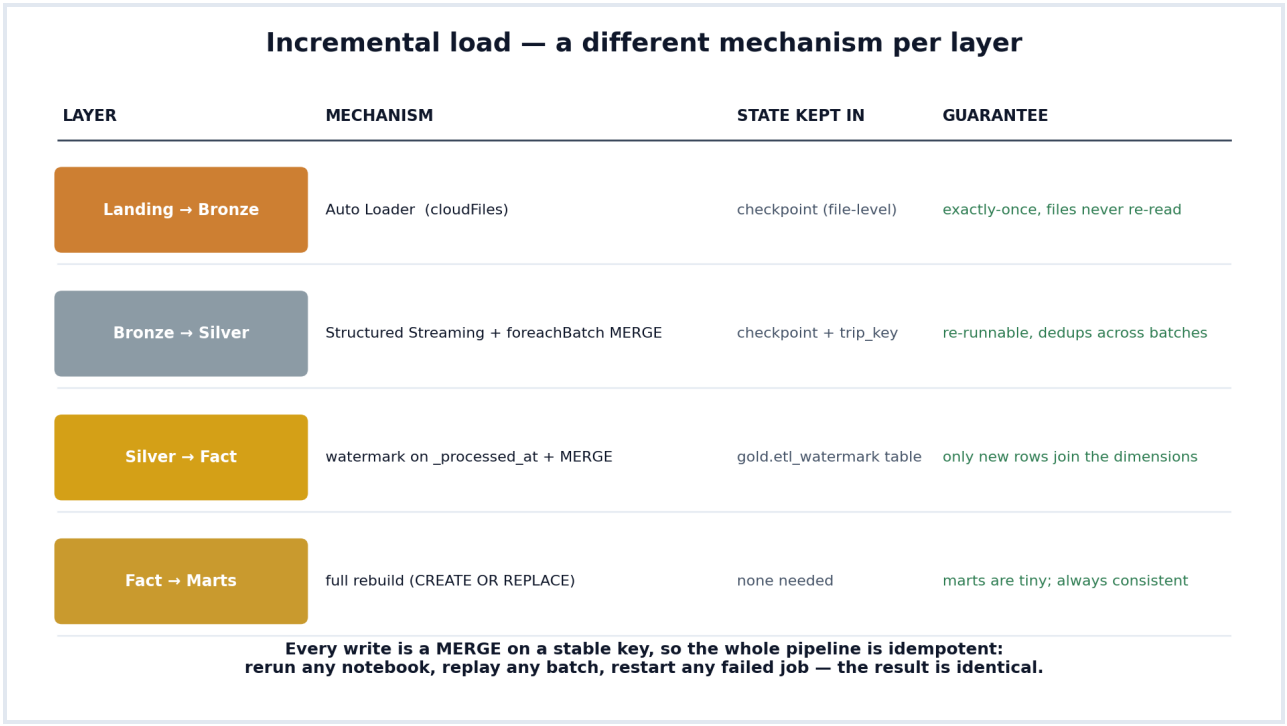


Figure 5 — Incremental mechanism by layer

What ties them together is that every write is a MERGE keyed on something stable. That single decision is what makes the pipeline idempotent: any notebook can be rerun, any batch replayed, any failed job restarted, and the result is identical. Without it, every retry becomes a correctness risk and every operational incident becomes a data incident.

7.1 What Delta ACID buys in practice

Property	What it prevents
Atomicity	A job that fails mid-merge leaves the table exactly as it was — no partially loaded fact table
Consistency	Schema enforcement rejects a malformed write instead of corrupting the table
Isolation	The dashboard querying fact_trips during a merge sees the snapshot that was current when its query began
Durability	The transaction log is the source of truth; DESCRIBE HISTORY gives a full audit trail
Time travel	A bad load is reversed with RESTORE TABLE … TO VERSION AS OF n rather than a restore from backup

8. Physical design: partitioning, bucketing and clustering

The requirement was to use partitioning or bucketing, whichever suits the data best. The honest answer is that both classic options are the wrong choice for the fact table, and it is worth setting out why.

8.1 Bucketing

Delta Lake on Databricks does not support Hive bucketing. The problem bucketing historically solved — avoiding a shuffle when joining two large tables on the bucket key — is now handled by adaptive query execution, dynamic file pruning and broadcast joins against the small dimensions in this model. Bucketing here would add rigidity and buy nothing.

8.2 Partitioning by pickup_date

This is the obvious candidate and it is a trap. At roughly 120,000 trips per day, one partition per day produces files far below the 128 MB to 1 GB range where Delta performs well; small-file overhead then dominates query time. It also hard-codes a single access path, so filtering by zone — the second most common predicate in the dashboard — still scans everything. And changing your mind later means rewriting the table.

The rule of thumb worth carrying away: do not partition a Delta table below roughly one terabyte unless the partition column is low cardinality and every partition will hold at least about a gigabyte.

8.3 Liquid clustering — the choice made here

```
CREATE TABLE gold.fact_trips ( ... )
USING DELTA
CLUSTER BY (pickup_date, pu_zone_key)
```

- Data skipping works on both predicates, not just one.
- File sizes are managed automatically; there is no small-file problem to police.
- Clustering keys can be changed later with ALTER TABLE … CLUSTER BY without rewriting the table.
- It replaces the OPTIMIZE … ZORDER BY maintenance cycle.

8.4 Where classic partitioning is still correct

gold.mart_daily_kpis is partitioned by year_month. The cardinality is low — one partition per month — the table is always filtered by date range, and partition pruning is exactly the right mechanism. Using both techniques in one project, each where it fits, is the point.

9. Orchestration

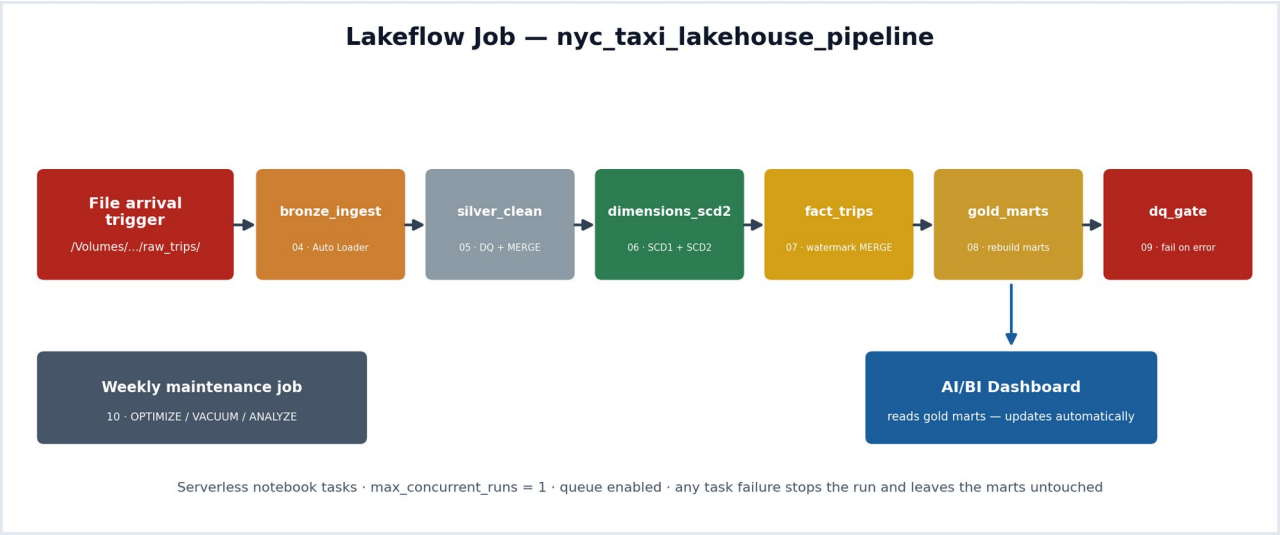


Figure 6 — Lakeflow job: trigger, task graph and downstream dashboard

The job is triggered by file arrival on the landing volume rather than by a clock. Databricks polls the path, waits for writes to settle, and starts the run. The practical difference is that the dashboard reflects new data minutes after it lands instead of at the next scheduled slot, and no compute is consumed on days when nothing arrives.

Task	Notebook	Responsibility
`bronze_ingest`	04	Auto Loader picks up new files and appends to bronze
`silver_clean`	05	Conform, score against DQ rules, quarantine failures, merge survivors
`dimensions_scd2`	06	Rebuild dim_date, merge SCD1 code dimensions, run the SCD2 merge
`fact_trips`	07	Watermark-based incremental load with surrogate key resolution
`gold_marts`	08	Rebuild the five marts and the health view
`dq_gate`	09	Nine assertions; raises and fails the run if any is violated

A separate weekly job runs OPTIMIZE, VACUUM with the safe 168-hour retention, and ANALYZE for statistics. Keeping maintenance out of the critical path means a slow compaction never delays the dashboard.

9.1 Failure behaviour

Tasks run in strict dependency order and max_concurrent_runs is set to one, so two file drops in quick succession queue rather than race. If any task fails, downstream tasks do not run — which means the gold marts, and therefore the dashboard, keep showing the last known-good numbers rather than a partial load. Because every stage is idempotent, the fix is simply to repair and rerun the job.

10. Serving layer: the dashboard

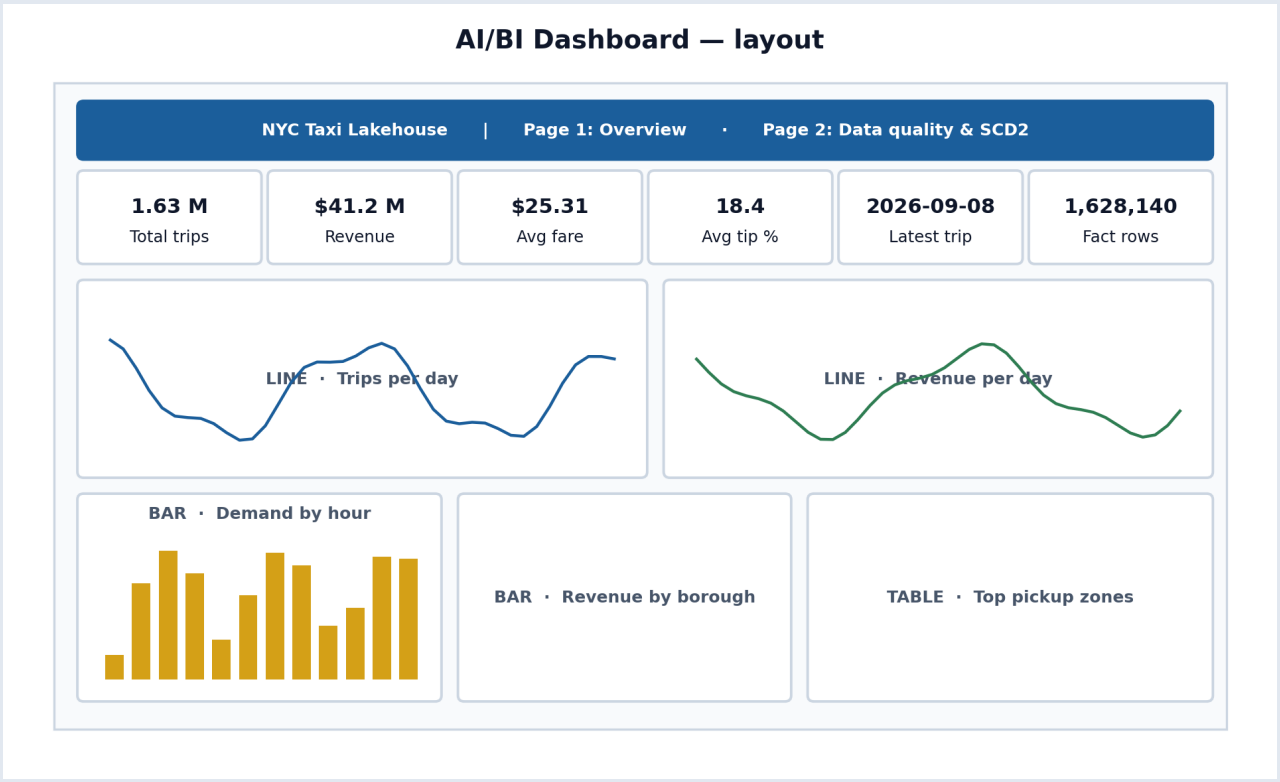


Figure 7 — AI/BI dashboard layout

The dashboard has two pages. The overview page carries six KPI tiles, daily trip and revenue trends, demand by hour of day, revenue by borough and a top-zones table. The second page is the one that usually gets the most attention in a review: it shows the pipeline health metrics — row counts at each layer, the DQ pass rate, quarantine volume — alongside the dim_zone version history, which is where the SCD Type 2 behaviour becomes visible rather than theoretical.

Every tile reads a gold mart. Because those marts are rebuilt by the fifth task of every run, the refresh chain from file arrival to updated dashboard requires no manual step at any point.

11. Governance and security

Control	Implementation
Namespace isolation	Separate schemas per medallion layer; the layer is visible in every fully qualified table name
Least privilege	Engineers hold ALL PRIVILEGES on bronze and silver; analysts hold SELECT on gold only
File-level governance	The landing volume (or external location) is a Unity Catalog securable with its own READ/WRITE VOLUME grants
Lineage	Captured automatically by Unity Catalog, column level, from landing volume through to dashboard
Auditability	Delta transaction log per table plus workspace audit logs; every load is attributable
PII	None present. The TLC dataset carries no rider identifiers; zone identifiers are coarse by design

```
GRANT USE CATALOG ON CATALOG nyc_taxi TO `analysts`;
GRANT USE SCHEMA, SELECT ON SCHEMA nyc_taxi.gold TO `analysts`;
GRANT ALL PRIVILEGES ON SCHEMA nyc_taxi.bronze TO `data_engineers`;
GRANT READ VOLUME, WRITE VOLUME ON VOLUME nyc_taxi.landing.files TO `data_engineers`;
```

12. Testing and validation

The quality gate runs as the last task and raises on any violation, which turns a silent data defect into a failed job.

#	Assertion	Failure would mean
1	fact_trips contains rows	The load silently produced nothing
2	No duplicate trip_key in the fact table	The merge key or dedup logic is broken
3	No orphan pickup zone keys	The point-in-time join is missing dimension coverage
4	No null date_key	dim_date does not span the loaded date range
5	Exactly one current row per zone	The SCD2 close-out step failed
6	No overlapping SCD2 validity windows	Effective dating is corrupt; point-in-time joins would fan out
7	No non-positive revenue in the fact table	A DQ rule was bypassed
8	DQ pass rate at or above 85 %	Upstream data has degraded materially
9	Gold marts are not empty	The mart rebuild silently produced nothing

Assertions 5 and 6 are the ones worth keeping in any SCD2 implementation. An overlapping validity window is invisible in a spot check and causes a point-in-time join to duplicate fact rows — inflating every measure downstream without ever throwing an error.

13. Free Edition constraints and mitigations

Constraint	Impact	Mitigation used
No storage credentials or external locations	Cannot attach your own S3 or ADLS	Unity Catalog volume as the landing zone; external syntax documented and ready in notebook 02

Constraint	Impact	Mitigation used
Serverless compute only	No cluster tuning, no long-running streams	Trigger.AvailableNow — streams start, drain and stop
Compute budget	Large volumes will exhaust the allowance	Batches sized at 120k rows/day; a single parameter reduces it
No always-on streaming	Continuous mode is impractical	File-arrival trigger on the job instead
DBFS mounts unavailable	Legacy paths do not resolve	All paths addressed through /Volumes/...

None of these change the shape of the solution. Moving to a paid workspace means running notebook 02, changing one path in the configuration notebook, and optionally converting the tables to external. The transformation logic, the data model and the orchestration are unaffected.

14. Running the proof of concept

Step	Action	Detail
1	Import the project	Upload the archive to the workspace, or clone the repository as a Git folder
2	Run notebook 01	Creates the catalog, four schemas, the landing volume and the directory structure
3	Run notebook 03	mode = initial, zone_version = 1 — writes 14 days of trip batches plus the zone snapshot
4	Run notebooks 04 to 09 in order	Each ends with a verification query; read the output rather than skipping ahead
5	Create the Lakeflow job	Paste jobs/nyc_taxi_job.json into the job editor and replace the notebook path prefix
6	Import the dashboard	dashboard/nyc_taxi_dashboard.lvdash.json, or build it from dashboard_queries.sql

14.1 The demonstration

The single test that exercises the whole design is this. Note the current dashboard numbers, then rerun notebook 03 with mode = incremental and zone_version = 2. That drops one new day of trips into the landing volume and, at the same time, a zone lookup in which four zones have changed attributes and two new zones have appeared.

- Auto Loader ingests only the new batch folder; the 1.8 million rows already in bronze are not re-read.
- Silver merges the new day in. Existing rows are untouched.
- dim_zone closes four rows and inserts four new versions, plus two first versions for the new zones.
- The fact table loads only rows past its watermark, and the point-in-time join sends the new day's trips to the new zone versions while yesterday's trips stay on the old ones.
- The marts rebuild, and the dashboard shows the extra day and the new version history without anyone touching it.

15. Limitations and next steps

15.1 Known limitations

- Trip data is synthetically generated to the official TLC schema. It is statistically plausible — two rush-hour peaks, realistic fare construction, deliberately injected defects — but it is not real. Pointing the pipeline at real TLC parquet files requires no code change.
- The zone lookup is an abridged subset of the official 265-row table, sufficient to demonstrate the SCD2 mechanics.
- Hard deletes in the reference data are not handled. A production SCD2 loader would also close out zones that disappear from the source snapshot.
- Surrogate keys contain gaps, because branch A rows that only perform an update still consume a row number. This is harmless for surrogate keys but would surprise someone expecting a dense sequence.

15.2 Natural extensions

- Add the green taxi and for-hire vehicle feeds as additional sources into the same silver and gold model.
- Convert the pipeline to Lakeflow Declarative Pipelines to get expectations, lineage and auto-recovery as declarations rather than code.
- Add a snapshot fact for driver or vendor daily aggregates alongside the transaction fact.
- Introduce a demand forecasting model using the hourly mart as a feature source.
- Promote the job definitions to an asset bundle deployed through CI with dev, staging and production targets.

Appendix A — Object inventory

Object	Type	Notes
`nyc_taxi.landing.files`	Volume	Landing zone; raw_trips, raw_zones, _checkpoints, _schemas
`nyc_taxi.bronze.trips_raw`	Delta table	Append only, CDF enabled, ingestion metadata
`nyc_taxi.bronze.zones_raw`	Delta table	Latest full zone snapshot
`nyc_taxi.silver.trips_clean`	Delta table	Clustered by (pickup_date, pu_location_id)
`nyc_taxi.silver.trips_quarantine`	Delta table	Rejected rows with dq_failures array
`nyc_taxi.silver.zone_stage`	Delta table	SCD2 source with hash_diff
`nyc_taxi.gold.fact_trips`	Delta table	Clustered by (pickup_date, pu_zone_key)
`nyc_taxi.gold.dim_zone`	Delta table	SCD Type 2
`nyc_taxi.gold.dim_date`	Delta table	Type 0, 2024–2027
`nyc_taxi.gold.dim_vendor / dim_payment_type / dim_rate_code`	Delta tables	Type 1, merged not overwritten
`nyc_taxi.gold.mart_daily_kpis`	Delta table	Partitioned by year_month
`nyc_taxi.gold.mart_hourly_demand / mart_zone_performance / mart_payment_mix / mart_vendor_scorecard`	Delta tables	Rebuilt every run

Object	Type	Notes
nyc_taxi.gold.vw_pipeline_health	View	Freshness and DQ tiles
nyc_taxi.gold.etl_watermark	Delta table	Incremental load bookmarks

Appendix B — Repository contents

Path	Purpose
notebooks/00_config.py	Catalog, schema, path and data-quality settings in one place
notebooks/01_setup_unity_catalog.py	Catalog, schemas, volume, grants
notebooks/02_external_location_optional.py	Storage credentials, external locations, external tables
notebooks/03_generate_landing_data.py	Source simulator with initial and incremental modes
notebooks/04_bronze_ingest.py	Auto Loader ingestion
notebooks/05_silver_clean.py	Cleaning, DQ rules, quarantine, dedup merge
notebooks/06_dimensions_scd2.py	dim_date, SCD1 dimensions, SCD2 dim_zone
notebooks/07_fact_trips.py	Incremental fact load with point-in-time joins
notebooks/08_gold_marts.py	Dashboard marts and health view
notebooks/09_data_quality_checks.py	Nine-assertion quality gate
notebooks/10_optimize_and_maintenance.py	OPTIMIZE, VACUUM, ANALYZE, time travel
jobs/nyc_taxi_job.json	Lakeflow job with a file-arrival trigger
jobs/nyc_taxi_job_scheduled.json	Same pipeline on a cron schedule
jobs/databricks.yml	Asset bundle definition
dashboard/nyc_taxi_dashboard.lvdash.json	Importable AI/BI dashboard
dashboard/dashboard_queries.sql	All dataset queries, to build the dashboard by hand
data/taxi_zone_lookup.csv, taxi_zone_lookup_v2.csv	Reference data, versions 1 and 2
docs/DESIGN_DECISIONS.md, docs/RUNBOOK.md	Rationale and operational procedures

Appendix C — Glossary

Term	Meaning
Medallion architecture	Layered refinement pattern — bronze (raw), silver (cleaned), gold (business-ready)
Auto Loader	Databricks incremental file ingestion source; tracks processed files in a checkpoint
MERGE	Delta operation combining insert, update and delete against a matching condition
Idempotent	Running an operation twice produces the same result as running it once
SCD Type 1	Overwrite in place; no history retained
SCD Type 2	New row per change with effective dating and a current flag; full history retained
Surrogate key	Meaningless integer key owned by the warehouse, one per version of a dimension row
hash_diff	Hash of tracked attributes used to detect change in a single comparison
Point-in-time join	Joining a fact to the dimension version that was in effect at the event time
Liquid clustering	Delta layout technique giving multi-column data skipping without fixed partitions
Watermark	Stored bookmark marking how far a downstream load has consumed its source
Quarantine	Table holding records that failed validation, retained with their failure reasons
Grain	The precise meaning of one row in a fact table